DBMS MINI PROJECT

Smart Restaurant Ordering, Reservation and Management System

SQL Database Design and Implementation

Database	PostgreSQL
Application Stack	Next.js + PostgreSQL + MongoDB
Document Scope	ER Diagram, Relational Schema, 3NF, DDL, Sample Data, Queries and Triggers
Project Type	DBMS Mini Project

1. Problem Statement

Problem Statement: Design and implement a database-driven restaurant management system that enables customers and restaurant staff to efficiently manage menu items, table reservations, food orders, payments and operational records while maintaining data consistency and integrity.

2. Problem Description

The proposed system is designed for restaurant customers, staff and administrators to simplify day-to-day restaurant operations. Customers can browse the menu, reserve tables, place food orders, make payments and provide feedback. Restaurant staff can manage table availability, reservations, orders and customer service, while administrators can maintain menu, staff and transaction records.

The SQL database stores structured and transactional data such as Customer, Staff, Restaurant Table, Reservation, Category, Menu Item, Order, Order Item and Payment. The project is designed so that MongoDB can later be used for flexible data such as reviews, customer preferences and order customizations. The core SQL database must support queries such as finding available tables for a specific capacity, retrieving a customer's order history, calculating an order bill and identifying the most popular menu items.

Target Users

- Customers – browse the menu, make reservations, place orders and view bills.
- Restaurant Staff – manage reservations, tables and order status.
- Administrators – manage menu categories, menu items, staff and restaurant records.

Core Entities

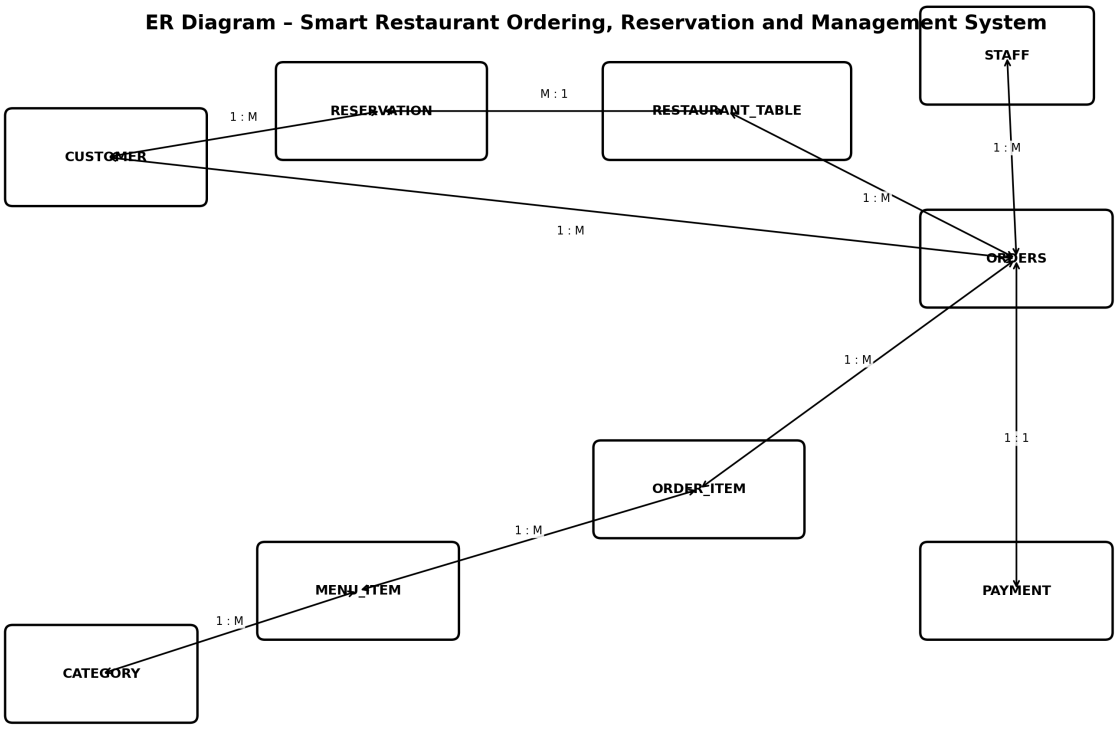
- Customer
- Staff
- Restaurant_Table
- Reservation
- Category
- Menu_Item
- Orders
- Order_Item
- Payment

Key Database Queries

- Find tables that are available for a required capacity and reservation time.
- Retrieve the complete order history of a particular customer.
- Display all items and calculate the total bill for an order.
- Find the most popular menu items based on quantity sold.

3. ER Diagram

The following ER diagram represents the main relational part of the restaurant system. Primary and foreign key details are given in the relational schema section.



4. Entity / Relationship Explanation

Relationship	Cardinality	Explanation
Customer — Reservation	1 : M	A customer can make multiple reservations; each reservation belongs to one customer.
Restaurant_Table — Reservation	1 : M	A table can be reserved at different dates/times; each reservation refers to one table.
Customer — Orders	1 : M	A customer can place many orders; each order is associated with one customer.
Staff — Orders	1 : M	A staff member can handle many orders; an order can be assigned to one staff member.
Restaurant_Table — Orders	1 : M	A table can be associated with multiple orders over time; an order may be associated with one table.
Orders — Order_Item	1 : M	One order contains one or more order-item records.
Menu_Item — Order_Item	1 : M	A menu item can appear in many order-item records.
Category — Menu_Item	1 : M	One category contains many menu items; each menu item belongs to one category.
Orders — Payment	1 : 1	Each order has at most one payment record in the current design.

Important many-to-many relationship: Orders and Menu_Item are logically M:N because an order contains multiple menu items and a menu item can occur in many orders. This is resolved using the associative entity Order_Item.

5. Relational Schema

Relation	Attributes
CUSTOMER	customer_id PK, name, email UNIQUE, phone UNIQUE, password, created_at
STAFF	staff_id PK, name, email UNIQUE, phone UNIQUE, role
RESTAURANT_TABLE	table_id PK, table_number UNIQUE, capacity, status
CATEGORY	category_id PK, category_name UNIQUE
MENU_ITEM	item_id PK, category_id FK, item_name, description, price, availability
RESERVATION	reservation_id PK, customer_id FK, table_id FK, reservation_date, reservation_time, guest_count, status
ORDERS	order_id PK, customer_id FK, table_id FK, staff_id FK, order_date, order_time, status, total_amount
ORDER_ITEM	order_id PK/FK, item_id PK/FK, quantity, unit_price, subtotal
PAYMENT	payment_id PK, order_id FK UNIQUE, amount, payment_method, payment_status, payment_time

Foreign Key Mapping

- Reservation.customer_id → Customer.customer_id
- Reservation.table_id → Restaurant_Table.table_id
- Menu_Item.category_id → Category.category_id
- Orders.customer_id → Customer.customer_id
- Orders.table_id → Restaurant_Table.table_id
- Orders.staff_id → Staff.staff_id
- Order_Item.order_id → Orders.order_id
- Order_Item.item_id → Menu_Item.item_id
- Payment.order_id → Orders.order_id

6. Normalization to 3NF

The relational design is structured to satisfy Third Normal Form (3NF). Normalization reduces redundancy and prevents insertion, update and deletion anomalies.

First Normal Form (1NF)

All attributes contain atomic values and there are no repeating groups. For example, multiple items in an order are not stored as item1, item2, item3 columns. They are represented as separate rows in ORDER_ITEM.

Second Normal Form (2NF)

Non-key attributes depend on the whole primary key. ORDER_ITEM has a composite primary key (order_id, item_id), and quantity and unit_price describe that particular order-item combination.

Third Normal Form (3NF)

There are no transitive dependencies among non-key attributes. For example, category_name is stored in CATEGORY rather than MENU_ITEM, and customer contact information is stored in CUSTOMER rather than ORDERS. Therefore, non-key attributes depend on the key, the whole key and nothing but the key.

Examples of Decomposition

- Customer details are separated from Orders using customer_id.
- Menu category information is separated from Menu_Item using category_id.
- The M:N relationship between Orders and Menu_Item is decomposed through Order_Item.
- Payment details are separated from Orders using order_id.

7. SQL DDL

The following DDL is written for PostgreSQL.

```
-- Create database separately if required:
-- CREATE DATABASE restaurant_management;

CREATE TABLE Customer (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    phone VARCHAR(15) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Staff (
    staff_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    phone VARCHAR(15) UNIQUE NOT NULL,
    role VARCHAR(30) NOT NULL
        CHECK (role IN ('Manager', 'Waiter', 'Chef', 'Cashier'))
);

CREATE TABLE Restaurant_Table (
    table_id SERIAL PRIMARY KEY,
    table_number INT UNIQUE NOT NULL,
    capacity INT NOT NULL CHECK (capacity > 0),
    status VARCHAR(20) DEFAULT 'Available'
        CHECK (status IN ('Available', 'Reserved', 'Occupied'))
);

CREATE TABLE Category (
    category_id SERIAL PRIMARY KEY,
    category_name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE Menu_Item (
    item_id SERIAL PRIMARY KEY,
    category_id INT NOT NULL,
    item_name VARCHAR(100) NOT NULL,
    description TEXT,
    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
```

```
    availability BOOLEAN DEFAULT TRUE,

    CONSTRAINT fk_menu_category
        FOREIGN KEY (category_id)
        REFERENCES Category(category_id)
        ON DELETE RESTRICT
);

CREATE TABLE Reservation (
    reservation_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    table_id INT NOT NULL,
    reservation_date DATE NOT NULL,
    reservation_time TIME NOT NULL,
    guest_count INT NOT NULL CHECK (guest_count > 0),
    status VARCHAR(20) DEFAULT 'Confirmed'
        CHECK (status IN ('Confirmed', 'Cancelled', 'Completed')),

    CONSTRAINT fk_reservation_customer
        FOREIGN KEY (customer_id)
        REFERENCES Customer(customer_id)
        ON DELETE CASCADE,

    CONSTRAINT fk_reservation_table
        FOREIGN KEY (table_id)
        REFERENCES Restaurant_Table(table_id)
        ON DELETE RESTRICT
);

CREATE TABLE Orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    table_id INT,
    staff_id INT,
    order_date DATE DEFAULT CURRENT_DATE,
    order_time TIME DEFAULT CURRENT_TIME,
    status VARCHAR(20) DEFAULT 'Placed'
        CHECK (status IN ('Placed', 'Preparing', 'Ready',
                          'Served', 'Completed', 'Cancelled')),
    total_amount DECIMAL(10,2) DEFAULT 0
        CHECK (total_amount >= 0),

    CONSTRAINT fk_order_customer
        FOREIGN KEY (customer_id)
        REFERENCES Customer(customer_id)
        ON DELETE RESTRICT,

    CONSTRAINT fk_order_table
        FOREIGN KEY (table_id)
        REFERENCES Restaurant_Table(table_id)
        ON DELETE SET NULL,

    CONSTRAINT fk_order_staff
        FOREIGN KEY (staff_id)
        REFERENCES Staff(staff_id)
        ON DELETE SET NULL
);

CREATE TABLE Order_Item (
    order_id INT NOT NULL,
```

```
    item_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
    subtotal DECIMAL(10,2)
        GENERATED ALWAYS AS (quantity * unit_price) STORED,

    PRIMARY KEY (order_id, item_id),

    CONSTRAINT fk_orderitem_order
        FOREIGN KEY (order_id)
        REFERENCES Orders(order_id)
        ON DELETE CASCADE,

    CONSTRAINT fk_orderitem_item
        FOREIGN KEY (item_id)
        REFERENCES Menu_Item(item_id)
        ON DELETE RESTRICT
);

CREATE TABLE Payment (
    payment_id SERIAL PRIMARY KEY,
    order_id INT UNIQUE NOT NULL,
    amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0),
    payment_method VARCHAR(20) NOT NULL
        CHECK (payment_method IN ('Cash', 'Card', 'UPI')),
    payment_status VARCHAR(20) DEFAULT 'Pending'
        CHECK (payment_status IN ('Pending', 'Completed', 'Failed')),
    payment_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_payment_order
        FOREIGN KEY (order_id)
        REFERENCES Orders(order_id)
        ON DELETE CASCADE
);
```

8. Sample SQL Data

```
INSERT INTO Customer (name, email, phone, password)
VALUES
('Rahul Sharma', 'rahul@gmail.com', '9876543210', 'pass123'),
('Ananya Singh', 'ananya@gmail.com', '9876543211', 'pass456'),
('Arjun Kumar', 'arjun@gmail.com', '9876543212', 'pass789');

INSERT INTO Staff (name, email, phone, role)
VALUES
('Vikram Rao', 'vikram@restaurant.com', '9000000001', 'Manager'),
('Priya Nair', 'priya@restaurant.com', '9000000002', 'Waiter'),
('Karan Das', 'karan@restaurant.com', '9000000003', 'Chef');

INSERT INTO Restaurant_Table (table_number, capacity, status)
VALUES
(1, 2, 'Available'),
(2, 4, 'Available'),
(3, 4, 'Available'),
(4, 6, 'Available'),
(5, 8, 'Available');
```

```
INSERT INTO Category (category_name)
VALUES
('Starters'),
('Main Course'),
('Desserts'),
('Beverages');

INSERT INTO Menu_Item
(category_id, item_name, description, price, availability)
VALUES
(1, 'Paneer Tikka',
 'Grilled Indian cottage cheese with spices', 220, TRUE),
(1, 'Chicken 65',
 'Spicy fried chicken appetizer', 250, TRUE),
(2, 'Veg Biryani',
 'Fragrant basmati rice with vegetables', 280, TRUE),
(2, 'Butter Chicken',
 'Creamy tomato-based chicken curry', 350, TRUE),
(3, 'Gulab Jamun',
 'Soft milk-solid dessert', 100, TRUE),
(4, 'Fresh Lime Soda',
 'Refreshing lime-based beverage', 80, TRUE);

INSERT INTO Reservation
(customer_id, table_id, reservation_date,
 reservation_time, guest_count)
VALUES
(1, 2, '2026-08-30', '19:30', 4);

INSERT INTO Orders (customer_id, table_id, staff_id)
VALUES (1, 2, 2);

-- Assuming the generated order_id is 1:
INSERT INTO Order_Item
(order_id, item_id, quantity, unit_price)
VALUES
(1, 1, 2, 220),
(1, 3, 1, 280),
(1, 6, 2, 80);

INSERT INTO Payment
(order_id, amount, payment_method, payment_status)
VALUES
(1, 880, 'UPI', 'Completed');
```

Expected order total for Order 1: 2×₹220 + 1×₹280 + 2×₹80 = ₹880. The order-total trigger in Section 10 can maintain this value automatically.

9. SQL Queries

Query 1 – Display complete menu

```
SELECT
    m.item_id,
    m.item_name,
    c.category_name,
    m.price,
```

```
    m.availability
FROM Menu_Item m
JOIN Category c
    ON m.category_id = c.category_id
ORDER BY c.category_name, m.item_name;
```

Query 2 – Retrieve customer order history

```
SELECT
    c.name,
    o.order_id,
    o.order_date,
    o.status,
    o.total_amount
FROM Customer c
JOIN Orders o
    ON c.customer_id = o.customer_id
WHERE c.customer_id = 1
ORDER BY o.order_date DESC;
```

Query 3 – Display details of an order

```
SELECT
    o.order_id,
    m.item_name,
    oi.quantity,
    oi.unit_price,
    oi.subtotal
FROM Orders o
JOIN Order_Item oi
    ON o.order_id = oi.order_id
JOIN Menu_Item m
    ON oi.item_id = m.item_id
WHERE o.order_id = 1;
```

Query 4 – Find the five most popular dishes

```
SELECT
    m.item_name,
    SUM(oi.quantity) AS total_quantity_sold
FROM Menu_Item m
JOIN Order_Item oi
    ON m.item_id = oi.item_id
JOIN Orders o
    ON oi.order_id = o.order_id
WHERE o.status <> 'Cancelled'
GROUP BY m.item_id, m.item_name
ORDER BY total_quantity_sold DESC
LIMIT 5;
```

Query 5 – Calculate completed revenue

```
SELECT
    COALESCE(SUM(amount), 0) AS total_revenue
FROM Payment
WHERE payment_status = 'Completed';
```

Query 6 – Find available tables for at least four people

```
SELECT *
FROM Restaurant_Table
WHERE status = 'Available'
  AND capacity >= 4
ORDER BY capacity;
```

Query 7 – Display pending/preparing orders

```
SELECT
    o.order_id,
    c.name AS customer_name,
    o.status,
    o.order_date,
    o.order_time
FROM Orders o
JOIN Customer c
    ON o.customer_id = c.customer_id
WHERE o.status IN ('Placed', 'Preparing')
ORDER BY o.order_date, o.order_time;
```

Query 8 – Show revenue by payment method

```
SELECT
    payment_method,
    SUM(amount) AS revenue
FROM Payment
WHERE payment_status = 'Completed'
GROUP BY payment_method
ORDER BY revenue DESC;
```

10. SQL Triggers

Triggers are used to enforce important business rules automatically at the database level. Two triggers are included: automatic maintenance of order totals and prevention of duplicate reservations for the same table, date and time.

Trigger 1 – Automatically Update Order Total

Whenever an Order_Item is inserted, updated or deleted, the corresponding Orders.total_amount is recalculated from the current order items.

```
CREATE OR REPLACE FUNCTION update_order_total()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        UPDATE Orders
        SET total_amount = (
            SELECT COALESCE(SUM(subtotal), 0)
            FROM Order_Item
            WHERE order_id = OLD.order_id
        )
        WHERE order_id = OLD.order_id;

        RETURN OLD;
```

```
    ELSE
        UPDATE Orders
        SET total_amount = (
            SELECT COALESCE(SUM(subtotal), 0)
            FROM Order_Item
            WHERE order_id = NEW.order_id
        )
        WHERE order_id = NEW.order_id;

        RETURN NEW;
    END IF;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_order_total
AFTER INSERT OR UPDATE OR DELETE
ON Order_Item
FOR EACH ROW
EXECUTE FUNCTION update_order_total();
```

Purpose: keeps the total amount in Orders synchronized with Order_Item records without requiring the application to manually recalculate the total after every change.

Trigger 2 – Prevent Reservation Conflict

Before inserting or updating a reservation, the trigger checks whether the selected table is already reserved for the same date and time.

```
CREATE OR REPLACE FUNCTION check_reservation_conflict()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM Reservation
        WHERE table_id = NEW.table_id
          AND reservation_date = NEW.reservation_date
          AND reservation_time = NEW.reservation_time
          AND status = 'Confirmed'
          AND reservation_id <> COALESCE(NEW.reservation_id, -1)
    ) THEN
        RAISE EXCEPTION
        'Table % is already reserved at the selected date and time',
        NEW.table_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_reservation_conflict
BEFORE INSERT OR UPDATE
ON Reservation
FOR EACH ROW
EXECUTE FUNCTION check_reservation_conflict();
```

Purpose: prevents two confirmed reservations from being created for the same table at the same date and time.

Trigger Test Cases

Trigger	Test	Expected Result
Order total	Insert an Order_Item record	Orders.total_amount is recalculated.
Order total	Delete an Order_Item record	Orders.total_amount is recalculated.
Reservation conflict	Book an already reserved table at the same date/time	The insert/update is rejected with an exception.

Submission Checklist

- Problem Statement and Problem Description completed.
- ER diagram completed with entities and cardinalities.
- Relational schema with primary and foreign keys documented.
- Normalization explained up to 3NF.
- PostgreSQL DDL statements prepared.
- Sample data prepared for execution.
- SQL queries prepared for core database requirements.
- Triggers prepared and test cases identified.
- MongoDB design can be added as a separate NoSQL section for the full-stack implementation.